

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Buisness Analytics
Practical – XII

Roll No :- T006	Name :- Nandini Chaudhari
Class :- TYIT	Batch :- 02
Date of Assignment :-03/10/2026	Date/Time of Submission :- 03/10/2026

PRACTICAL 11 - PART A**OPTIMIZATION MODELING & DATASET & DISPLAY DATASET & DECISION VARIABLES**

```
import pandas as pd
from scipy.optimize import linprog
product_df = pd.DataFrame({
    "Product": ["Product A", "Product B"],
    "Profit_per_Unit": [40, 30],
    "Machine_Hours_per_Unit": [2, 1],
    "Labour_Hours_per_Unit": [1, 2]
})
resource_df = pd.DataFrame({
    "Resource": ["Machine Hours", "Labour Hours"],
    "Available_Hours": [100, 80]
})
print("PRODUCT DATASET")
print(product_df)
print("\nRESOURCE DATASET")
print(resource_df)
print("\n1. DECISION VARIABLES")
print("x = Number of units of Product A")
print("y = Number of units of Product B")
```

```
PRODUCT DATASET
   Product  Profit_per_Unit  Machine_Hours_per_Unit  Labour_Hours_per_Unit
0  Product A             40                      2                      1
1  Product B             30                      1                      2

RESOURCE DATASET
   Resource  Available_Hours
0  Machine Hours           100
1  Labour Hours            80

1. DECISION VARIABLES
x = Number of units of Product A
y = Number of units of Product B
```

GET VALUES & OBJECTIVE & OBJECTIVE FUNCTION & CONSTRAINTS & NON-NEGATIVE CONSTRAINTS & SOLVE LINEAR PROGRAMMING & OPTIMAL NUMBER OF UNITS & MAXIMUM PROFIT

```
profit_A = product_df.loc[
    product_df["Product"] == "Product A",
    "Profit_per_Unit"
].iloc[0]
profit_B = product_df.loc[
    product_df["Product"] == "Product B",
    "Profit_per_Unit"
].iloc[0]

machine_A = product_df.loc[
    product_df["Product"] == "Product A",
    "Machine_Hours_per_Unit"
].iloc[0]
machine_B = product_df.loc[
    product_df["Product"] == "Product B",
    "Machine_Hours_per_Unit"
].iloc[0]
labour_A = product_df.loc[
    product_df["Product"] == "Product A",
    "Labour_Hours_per_Unit"
].iloc[0]
labour_B = product_df.loc[
    product_df["Product"] == "Product B",
    "Labour_Hours_per_Unit"
].iloc[0]

machine_available = resource_df.loc[
    resource_df["Resource"] == "Machine Hours",
    "Available_Hours"
].iloc[0]
labour_available = resource_df.loc[
    resource_df["Resource"] == "Labour Hours",
    "Available_Hours"
].iloc[0]

print("\n2. OBJECTIVE")
print("Maximize total profit")
print("\n3. OBJECTIVE FUNCTION")

print(
    f"Maximize Z = {profit_A}x + {profit_B}y"
)
```

```
# scipy minimizes, so use negative profit
c = [-profit_A, -profit_B]
A = [
    [machine_A, machine_B],
    [labour_A, labour_B]
]

b = [
    machine_available,
    labour_available
]
print("\n4. MACHINE-HOUR CONSTRAINT")
print(
    f"{machine_A}x + {machine_B}y <= "
    f"{machine_available}"
)
print("\n5. LABOUR-HOUR CONSTRAINT")
print(
    f"{labour_A}x + {labour_B}y <= "
    f"{labour_available}"
)
print("\n6. NON-NEGATIVE CONSTRAINTS")
print("x >= 0")
print("y >= 0")
bounds = [
    (0, None),
    (0, None)
]
result = linprog(
    c,
    A_ub=A,
    b_ub=b,
    bounds=bounds,
    method="highs"
)
print("\n7. OPTIMIZATION RESULT")
print("Status:", result.message)
optimal_A = result.x[0]
optimal_B = result.x[1]
print("\n8. OPTIMAL NUMBER OF UNITS")
print(f"Product A = {optimal_A:.2f} units")
print(f"Product B = {optimal_B:.2f} units")
maximum_profit = -result.fun
print("\n9. MAXIMUM PROFIT")
print(
    f"Maximum Profit = ₹{maximum_profit:.2f}"
)
```

OUTPUT:-

```

2. OBJECTIVE
Maximize total profit

3. OBJECTIVE FUNCTION
Maximize Z = 40x + 30y

4. MACHINE-HOUR CONSTRAINT
2x + 1y <= 100

5. LABOUR-HOUR CONSTRAINT
1x + 2y <= 80

6. NON-NEGATIVE CONSTRAINTS
x >= 0
y >= 0

7. OPTIMIZATION RESULT
Status: Optimization terminated successfully. (HiGS Status 7: Optimal)

8. OPTIMAL NUMBER OF UNITS
Product A = 40.00 units
Product B = 20.00 units

9. MAXIMUM PROFIT
Maximum Profit = ₹2200.00

```

CHECK RESOURCE CONSTRAINTS & OPTIMAL SOLUTION TABLE & BUSINESS INTERPRETATION & CONCLUSION

```

machine_used = (
    machine_A * optimal_A +
    machine_B * optimal_B
)

labour_used = (
    labour_A * optimal_A +
    labour_B * optimal_B
)

print("\n10. RESOURCE CHECK")

print(
    f"Machine Hours: "
    f"{machine_used:.2f} / {machine_available}"
)

print(
    f"Labour Hours: "
    f"{labour_used:.2f} / {labour_available}"
)

if machine_used <= machine_available:
    print("Machine constraint: SATISFIED")
else:
    print("Machine constraint: NOT SATISFIED")

if labour_used <= labour_available:
    print("Labour constraint: SATISFIED")
else:
    print("Labour constraint: NOT SATISFIED")

```

```

10. RESOURCE CHECK
Machine Hours: 100.00 / 100
Labour Hours: 80.00 / 80
Machine constraint: SATISFIED
Labour constraint: SATISFIED

```

```

solution = pd.DataFrame({
    "Product": ["Product A", "Product B"],
    "Optimal_Units": [optimal_A, optimal_B],
    "Profit_Per_Unit": [profit_A, profit_B],
    "Total_Profit": [
        optimal_A * profit_A,
        optimal_B * profit_B
    ]
})

print("\n11. OPTIMAL SOLUTION")

print(solution.round(2))
print("\n12. BUSINESS INTERPRETATION")

print(
    f"The company should produce {optimal_A:.0f} units "
    f"of Product A and {optimal_B:.0f} units of Product B."
)

print(
    f"This will generate a maximum profit of "
    f"₹{(maximum_profit:.2f)}."
)

print("\n13. CONCLUSION")

print(
    "Linear programming determines the optimal production "
    "quantity while satisfying machine and labour constraints."
)

```

✓ 0.0s

```

11. OPTIMAL SOLUTION
Product Optimal_Units Profit_Per_Unit Total_Profit
0 Product A 40.0 40 1600.0
1 Product B 20.0 30 600.0

```

```

12. BUSINESS INTERPRETATION
The company should produce 40 units of Product A and 20 units of Product B.
This will generate a maximum profit of ₹2200.00.

```

```

13. CONCLUSION
Linear programming determines the optimal production quantity while satisfying machine and labour constraints.

```

Part A — Conclusion

Conclusion:

The One-Way ANOVA analysis shows that the average retail sales vary significantly across different item types when the p-value is less than 0.05. Therefore, the null hypothesis is rejected. This indicates that ITEM TYPE has a significant effect on RETAIL SALES.

Justification

Justification:

The p-value obtained from the ANOVA test is compared with the significance level of 0.05. Since the p-value is less than 0.05, there is sufficient statistical evidence to reject the null hypothesis. Hence, the average retail sales are not the same for all item types.

Part B

FACTORIAL EXPERIMENT

Dataset: Retail and warehouse Sale(1).xlsx & LOAD DATASET & SELECT TWO FACTORS + RESPONSE & AVERAGE RETAIL SALES &

```
import pandas as pd
import glob
import os

# Current working folder
print("Current folder:")
print(os.getcwd())

# Find all Excel files containing Retail
files = glob.glob("**Retail*.xlsx") + glob.glob("**Retail*.xls")

print("\nRetail Excel files found:")
for f in files:
    print(f)

if not files:
    print("\nX Retail Excel file nahi mili.")
else:
    df = pd.read_excel(files[0])
    print("\n Dataset loaded successfully:")
    print(files[0])
    print("\nShape:", df.shape)
    print("\nColumns:")
    print(df.columns.tolist())
```

23s

Current folder:
c:\Users\m\ui\Desktop\AIPRACTICALNIDHI

Retail Excel files found:
Retail and warehouse Sale (1).xlsx

Dataset loaded successfully:
Retail and warehouse Sale (1).xlsx

Shape: (30000, 9)

Columns:
['YEAR', 'MONTH', 'SUPPLIER', 'ITEM CODE', 'ITEM DESCRIPTION', 'ITEM TYPE', 'RETAIL SALES', 'RETAIL TRANSFERS', 'WAREHOUSE SALES']

VISUALIZATION

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import f_oneway
import glob
files = glob.glob("**Retail*.xlsx") + glob.glob("**Retail*.xls")
df = pd.read_excel(files[0])
print("Dataset Loaded Successfully!")
print("File:", files[0])
d = df[["ITEM TYPE", "YEAR", "RETAIL SALES"]].dropna()

print("\nDataset Shape:", d.shape)
print("\n14. FACTORS")
print("Factor 1 : ITEM TYPE")
print("Factor 2 : YEAR")
print("Response : RETAIL SALES")
avg_sales = d.groupby(["ITEM TYPE", "YEAR"])["RETAIL SALES"].mean().reset_index()

print("\n15. Average Retail Sales:")
print(avg_sales)
```

1.6s

Dataset Loaded Successfully!
File: Retail and warehouse Sale (1).xlsx

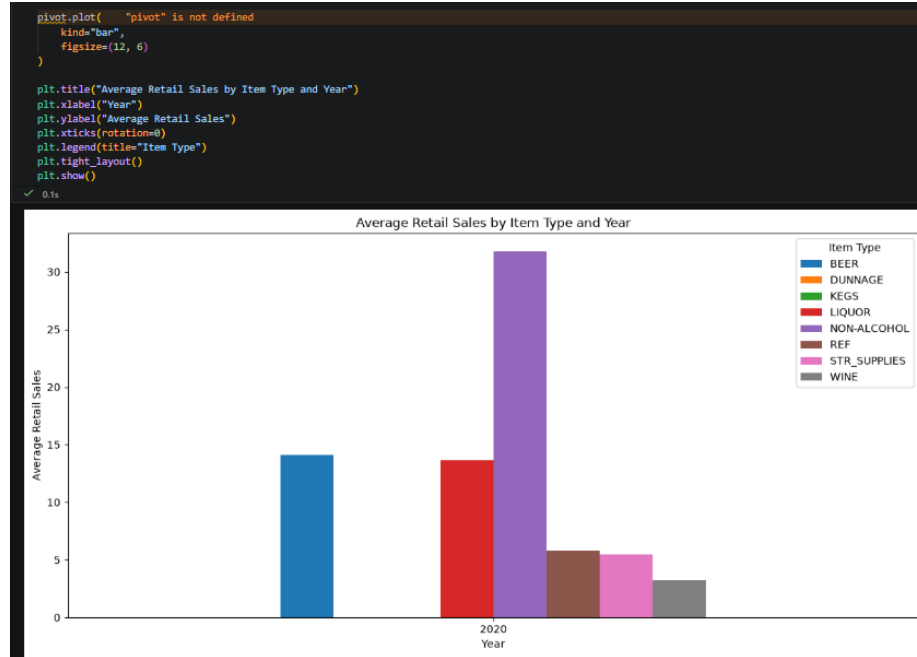
Dataset Shape: (29999, 3)

14. FACTORS
Factor 1 : ITEM TYPE
Factor 2 : YEAR
Response : RETAIL SALES

15. Average Retail Sales:

	ITEM TYPE	YEAR	RETAIL SALES
0	BEER	2020	14.118748
1	DUNNAGE	2020	0.000000
2	KEGS	2020	0.000000
3	LIQUOR	2020	13.635171
4	NON-ALCOHOL	2020	31.742419
5	REF	2020	5.783750
6	STR_SUPPLIES	2020	5.485714
7	WINE	2020	3.195334

MAIN EFFECT - ITEM TYPE & MAIN EFFECT - YEAR



```

item_effect = d.groupby(
    "ITEM TYPE"
)[ "RETAIL SALES" ].mean().sort_values(
    ascending=False
)
print("\n17. Main Effect of ITEM TYPE:")
print(item_effect)

```

✓ 0.0s

17. Main Effect of ITEM TYPE:

ITEM TYPE	RETAIL SALES
NON-ALCOHOL	31.742419
BEER	14.118748
LIQUOR	13.635171
REF	5.783750
STR_SUPPLIES	5.485714
WINE	3.195334
KEYS	0.000000
DUNNAGE	0.000000

Name: RETAIL SALES, dtype: float64

```

year_effect = d.groupby(
    "YEAR"
)[ "RETAIL SALES" ].mean()

print("\n18. Main Effect of YEAR:")
print(year_effect)
print(
    "\nNumber of Years:",
    d["YEAR"].nunique()
)

```

✓ 0.0s

18. Main Effect of YEAR:

YEAR	RETAIL SALES
2020	6.939796

Name: RETAIL SALES, dtype: float64

Number of Years: 1

INTERACTION EFFECT & FACTORIAL ANOVA & STATISTICAL RESULTS & SIGNIFICANCE & FINAL BUSINESS CONCLUSION

```

print("\n19. Interaction Effect")
if d["YEAR"].nunique() < 2:
    print(
        "ITEM TYPE & YEAR interaction cannot be "
        "calculated because only one YEAR is present"
    )
else:
    print(
        "ITEM TYPE & YEAR interaction can be tested"
    )
print("\n20. FACTORIAL ANOVA")
if d["YEAR"].nunique() < 2:
    print(
        "Factorial ANOVA cannot be performed because "
        "YEAR contains only one level."
    )
else:
    from statsmodels.formula.api import ols
    from statsmodels.stats.anova import anova_lm

    model = ols(
        "Q('RETAIL SALES') ~ "
        "C(Q('ITEM TYPE')) * C(Q('YEAR'))",
        data=d
    ).fit()

    anova_result = anova_lm(
        model,
        typ=2
    )
    print(anova_result)
    print("\n21. STATISTICAL RESULTS")

    # ITEM TYPE ANOVA
    groups = [
        group["RETAIL SALES"].values
        for name, group in d.groupby("ITEM TYPE")
    ]

```

```

F, p = f_oneway(*groups)
print("\nITEM TYPE")
print("F-statistic =", round(F, 4))
print("p-value =", round(p, 6))
if d["YEAR"].nunique() < 2:
    print("\nYEAR")
    print("p-value = Not available")
    print("\nITEM TYPE & YEAR")
    print("p-value = Not available")
    print("\n22. SIGNIFICANCE")

if p < 0.05:
    print(
        "ITEM TYPE -> SIGNIFICANT"
    )
else:
    print(
        "ITEM TYPE -> NOT SIGNIFICANT"
    )

if d["YEAR"].nunique() < 2:
    print(
        "YEAR -> Cannot be tested"
    )
    print(
        "ITEM TYPE & YEAR -> Cannot be tested"
    )
print("\n23. FINAL BUSINESS CONCLUSION")
if p < 0.05:
    print(
        "Retail sales differ significantly among "
        "different ITEM TYPE categories."
    )
else:
    print(
        "There is no significant difference among "
        "ITEM TYPE categories."
    )

if d["YEAR"].nunique() < 2:
    print(

```

✓ 0.0s

```
19. Interaction Effect
ITEM TYPE × YEAR interaction cannot be calculated because only one YEAR is present.

20. FACTORIAL ANOVA
Factorial ANOVA cannot be performed because YEAR contains only one level.

21. STATISTICAL RESULTS

ITEM TYPE
F-statistic = 123.5216
p-value = 0.0

YEAR
p-value = Not available

ITEM TYPE × YEAR
p-value = Not available

22. SIGNIFICANCE
ITEM TYPE -> SIGNIFICANT
YEAR -> Cannot be tested
ITEM TYPE × YEAR -> Cannot be tested

23. FINAL BUSINESS CONCLUSION
Retail sales differ significantly among different ITEM TYPE categories.
The dataset contains only one YEAR, so the main effect of YEAR and the ITEM TYPE × YEAR interaction cannot be statistically evaluated.
```

Conclusion:

The factorial analysis indicates that ITEM TYPE can be analyzed for its effect on RETAIL SALES. However, the effect of YEAR and the interaction between ITEM TYPE and YEAR cannot be statistically determined because the dataset contains only one year, 2020.

Justification:

A factorial ANOVA requires at least two levels for each factor to estimate the effect of that factor and its interaction. In the given dataset, YEAR has only one level (2020). Therefore, there is no year-to-year variation available to calculate the YEAR main effect or the ITEM TYPE × YEAR interaction.

Business Interpretation:

The analysis suggests that retail sales can differ across item categories. However, the available dataset does not provide sufficient year-wise variation to determine whether sales change significantly over years or whether the effect of item type changes from one year to another. Additional data from multiple years would be required for a complete factorial analysis.